COSC204 Computer Systems

Lecture 18: Processes sharing and threads

Zhiyi Huang

School of Computing, University of Otago

Overview

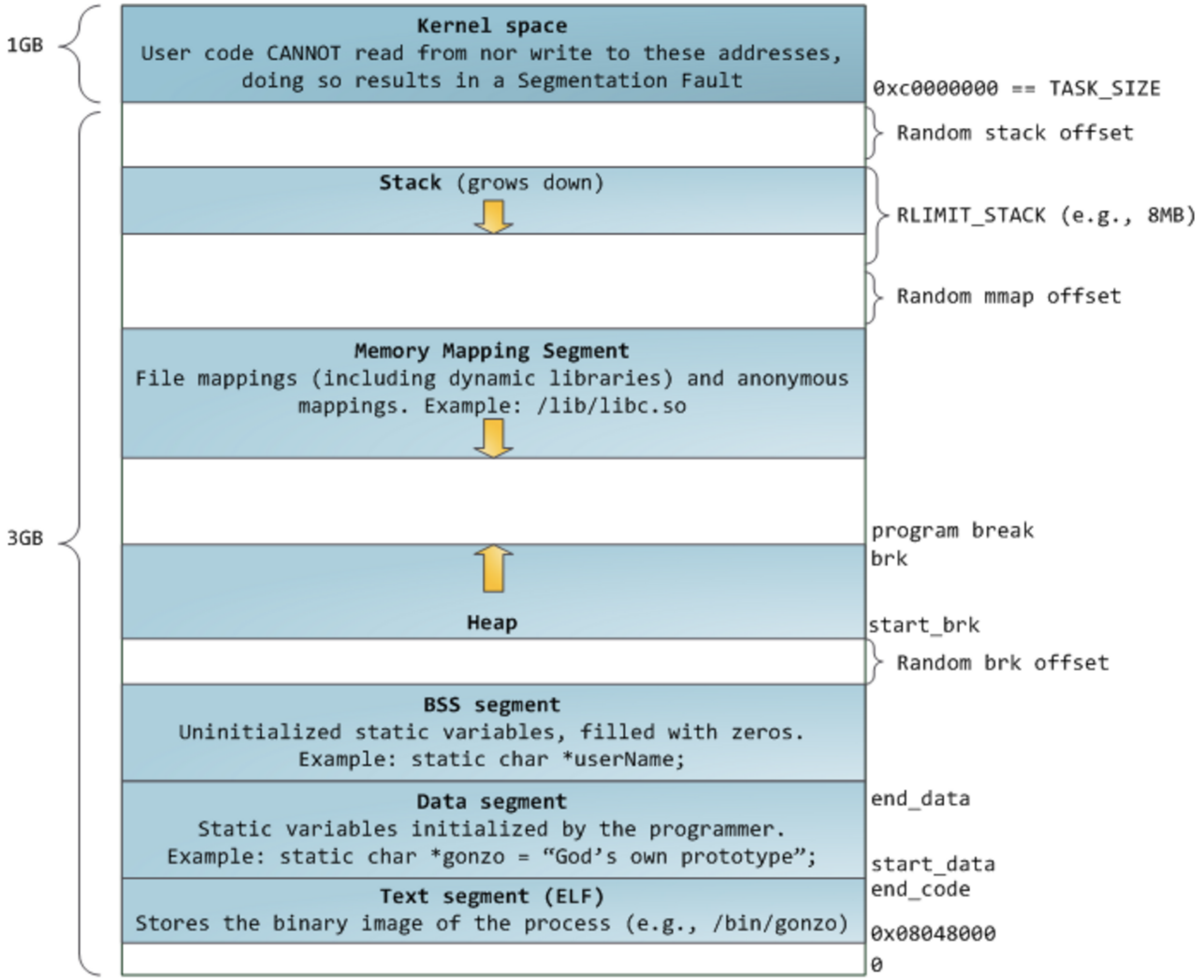
Last lecture:

- POSIX
- Cooperating processes
- Creating a child process
- Communication between parent and child processes
- Signals and pipes for inter-process communication

This lecture:

- Shared memory for processes
- Threads

Memory space layout for a 32-bit Linux process

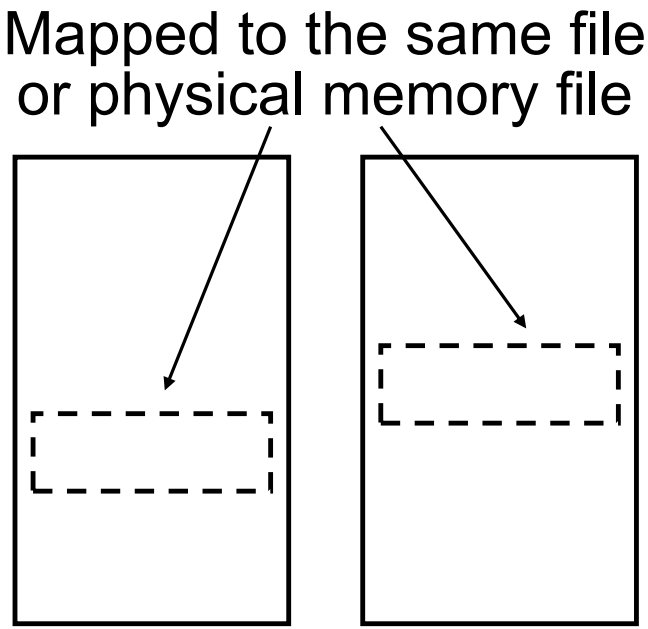


(image from Anatomy of a Program in Memory)

Shared memory: straightforward data sharing

The memory mapping system call `mmap()` modifies the memory layout of a process. It can also be used to allow two processes to *share* memory regions.

- In the figure below, two processes share a physical memory region by mapping the same resource into their own address spaces.



Examples using `mmap()`

Shared memory can be achieved by two processes `mmap`ping the same file or using `fork()` after `mmap()`.

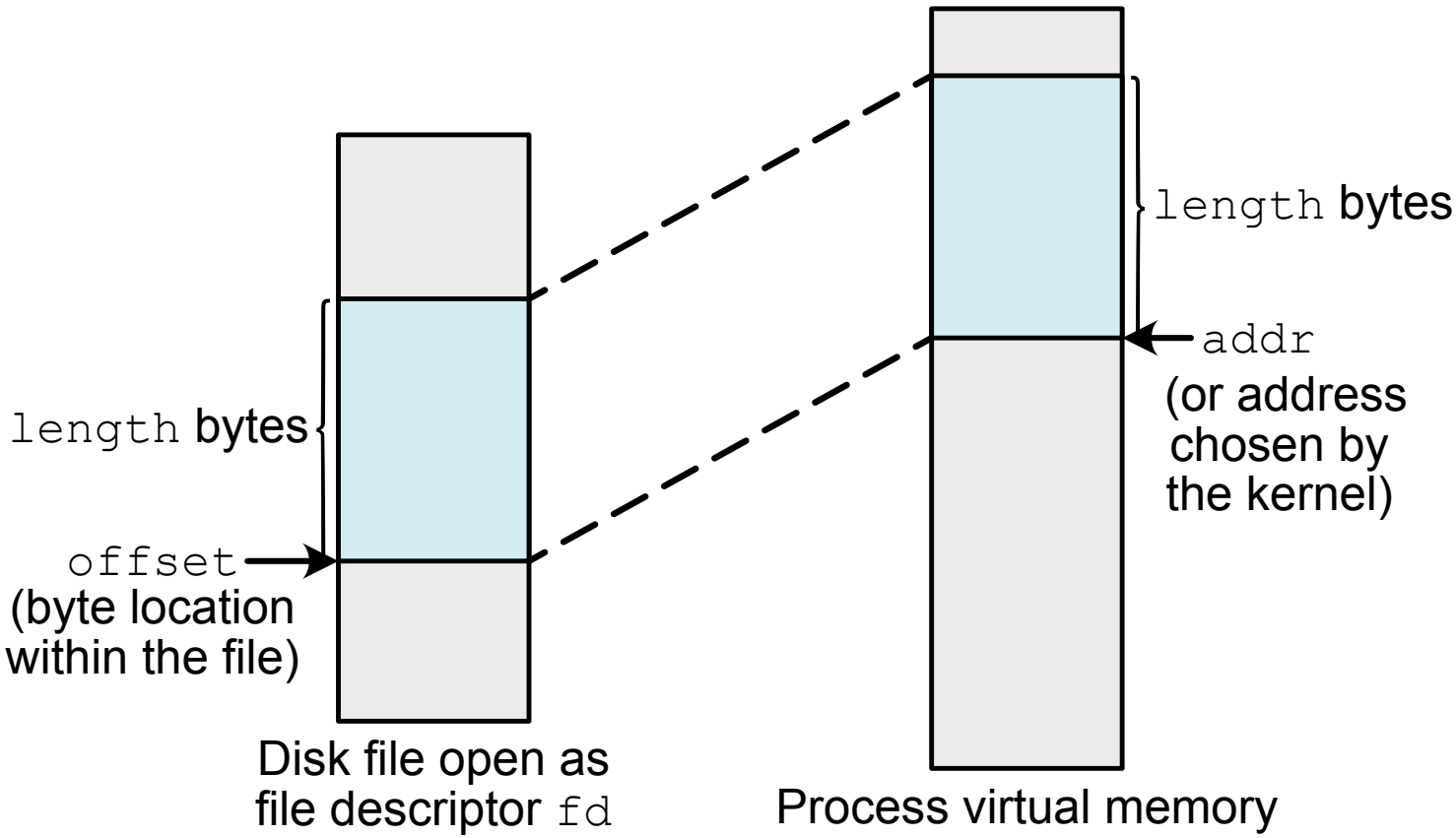
```
1 random_fd = open("/home/dme/test/zero", O_RDWR);
2 ptr = mmap(NULL, 10240, PROT_READ|PROT_WRITE,
3            MAP_SHARED|MAP_FILE, random_fd, 0);
```

Private mapping is often used to set up new memory sections.

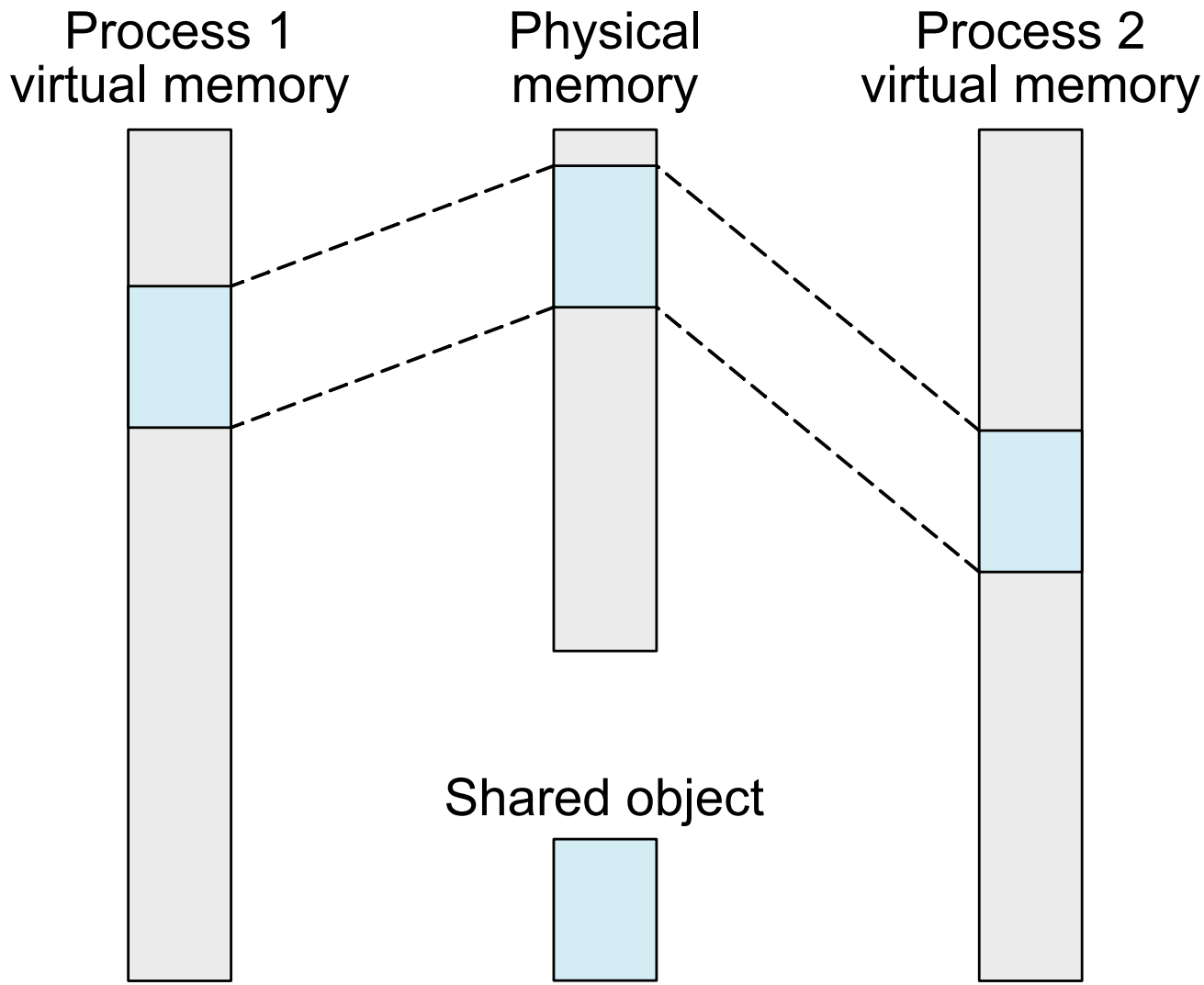
```
1 fd = open("mmap_test_file", O_RDWR);
2 ptr = mmap(NULL, 10240, PROT_READ|PROT_WRITE,
3            MAP_PRIVATE|MAP_FILE, fd, 0);
```

Private mapping for I/O performance

```
1 void *mmap(void *addr, size_t length, int prot,
2             int flags, int fd, off_t offset)
```



Shared mapping for process communication



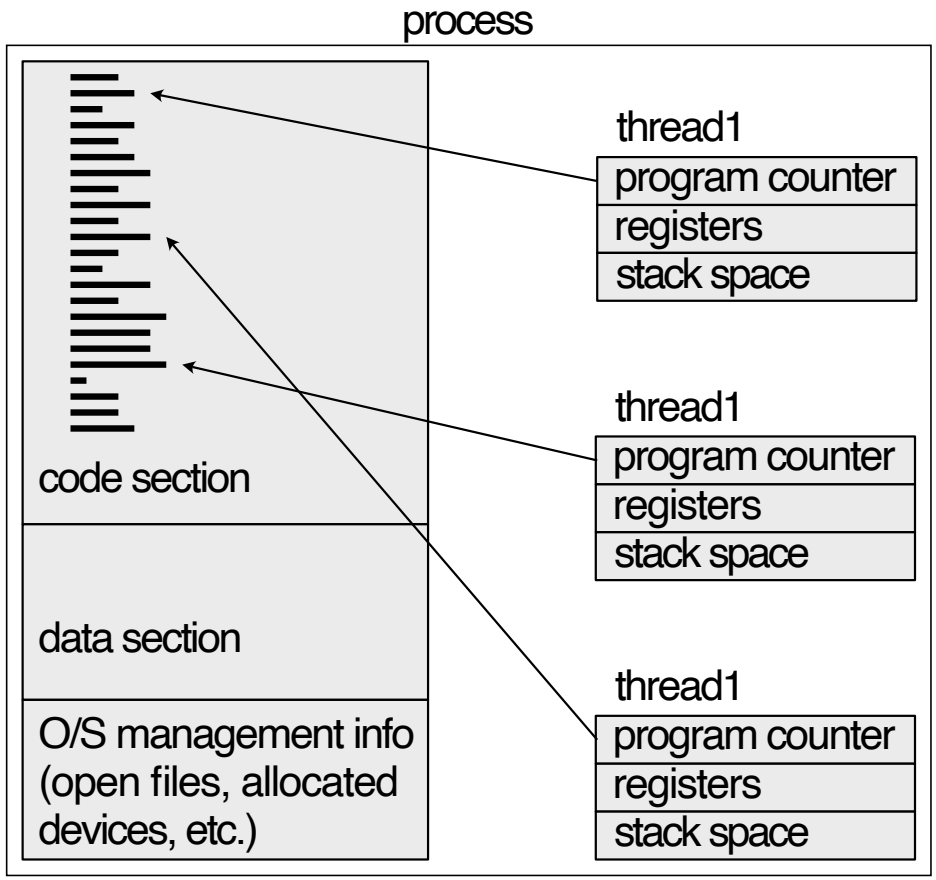
Threads are processes that share many resources

Processes can share data by sharing memory regions.

Many operating systems support a special kind of process called a thread to allow convenient data-sharing.

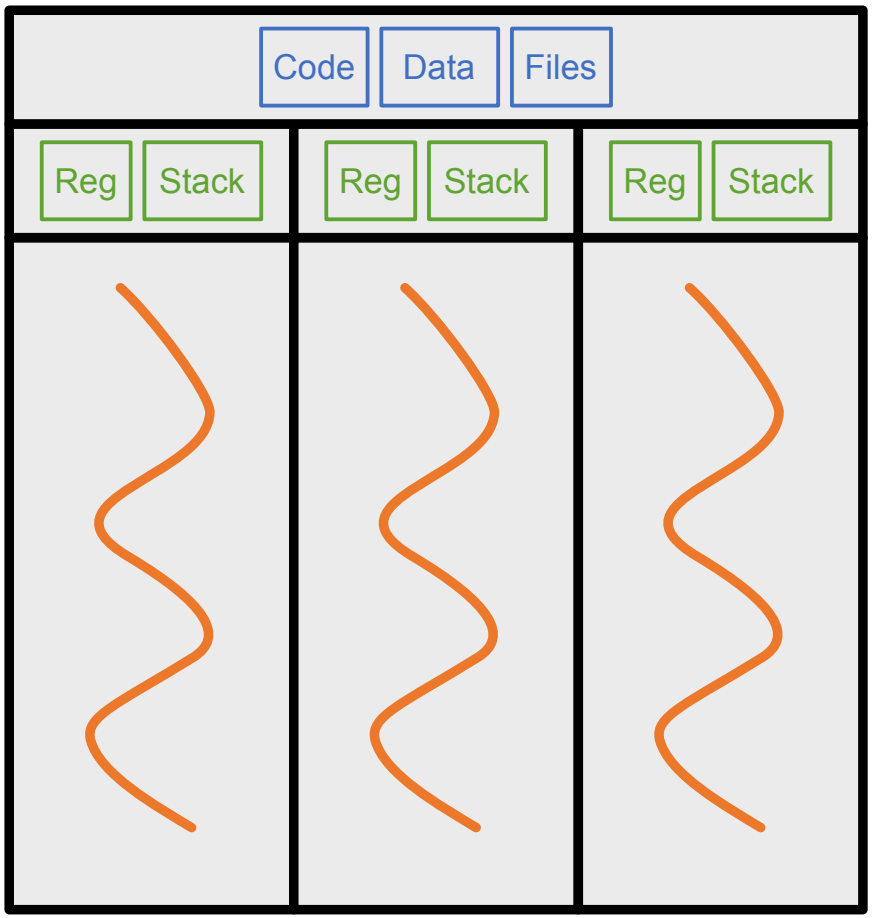
- A standard process has a data section, code section, a program counter, a register set, a stack space, information about open files, allocated I/O devices, etc.
- Threads within a single process use the same code section, data section, etc. However, they have their own program counter, registers and stack space.
- Thus, threads are not strongly isolated from each other: good for speed; bad for safety.

Depicting threads as lightweight processes

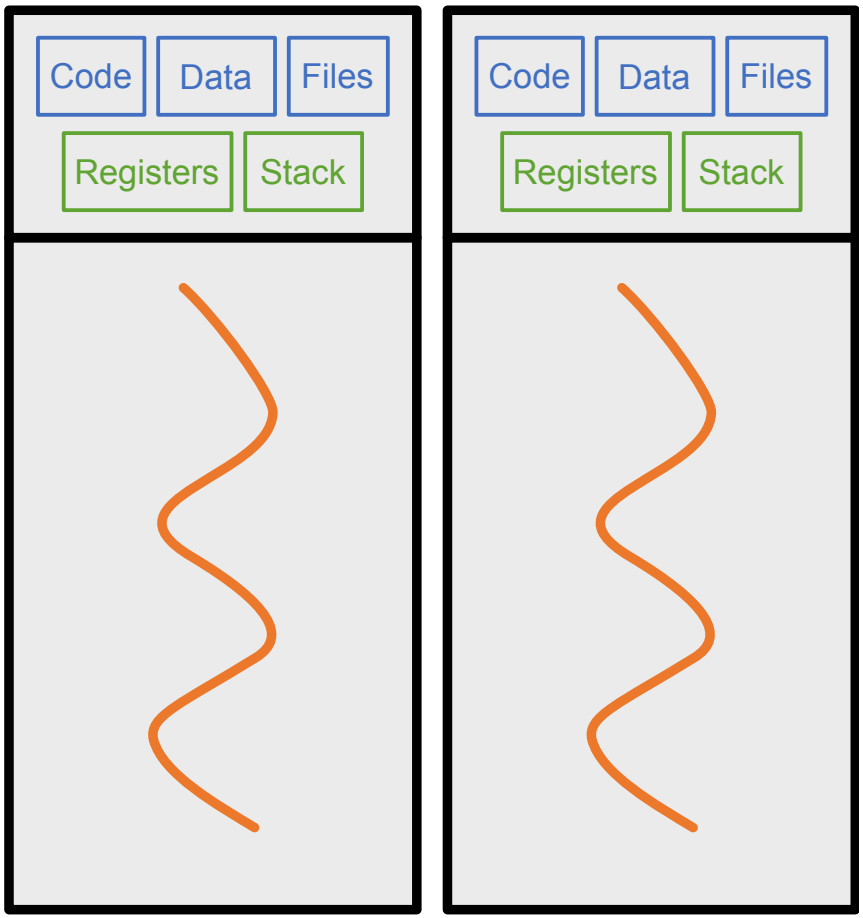


Reiterating: a thread is a type of process that shares the code section, data section, and O/S housekeeping information with other such lightweight processes (i.e., threads).

Multithreading versus multiprocessing



Multithreading



Multiprocessing

Why use threads instead of processes?

- **Responsiveness**—Multithreading an interactive application may allow a program to continue running, even if part of it is waiting.
- **Resource sharing**—Communication between threads is easy: there is no need to set up shared memory as for processes.
- **Economy**—Thread creation is easier. Switching between threads is faster than between (heavyweight) processes, because there is less CPU context to change.
- **Scalability**—Easy to work on multiprocessor architectures.

Pthreads—threads defined by the POSIX standard

POSIX standard specifies a set of functions for threaded programming.

- `pthread_create()`: create a thread
- `pthread_exit()`: terminate a thread
- `pthread_join()`: wait for a thread to terminate
- `pthread_detach()`: detach thread: won't wait for it in `_join()`
- `pthread_self()`: get the thread ID of the current thread
- `pthread_equal()`: compare if two threads are the same
- `pthread_cancel()`: send a cancellation request to a thread

Pthread example

```
1  void *myThreadFn(void *vargp)
2  {
3      sleep(1);
4      printf("Hello from Thread \n");
5      return NULL;
6  }
7
8  int main(void)
9  {
10     pthread_t thread_id;
11     printf("Before Thread\n");
12     pthread_create(&thread_id, NULL, myThreadFn, NULL);
13     pthread_join(thread_id, NULL);
14     printf("After Thread\n");
15     exit(EXIT_SUCCESS);
16 }
```

Servers may spawn threads to handle requests

A web server is a process that provides data over the web to requesting clients (possibly a very large number of them).

The web server might create threads to deal with each separate request that arrives: this is a typical application of multithreading.

Readings

For this lecture, the relevant material is in Chapter 3 (Sections 4–5) and Chapter 4 (Sections 1–4) of Silberschatz *et al*.